

Supervised assets

Issuer-freezable assets on Sequentia: what consensus can enforce, what it cannot, and who covers the difference. 2026-08-12, for Andreas and Alberto.

Proposal: add a declared asset class, *supervised*, whose issuer can freeze holders by consensus rule, and pair it with freeze-awareness in the Lightning and DEX implementations. Freezability is committed into the asset id at issuance, so it is permanent, visible before acquisition, and cannot be added to an existing asset.

The split is not a compromise, it is a boundary. Consensus can freeze what it can see and attribute: ordinary on-chain holdings, and resting covenant orders. It cannot freeze a Lightning balance, and no version of this rule ever will, because a channel's value lives in a shared output and in off-chain state that consensus cannot attribute to either party. That half must be enforced by the Lightning implementation as policy, which is exactly the division Andreas described.

This matches how the same problem is handled elsewhere: blacklisting an Ethereum address does not reach the holder's balance on a rollup or at an exchange either, and Circle covers those by working with the operator rather than with the chain.

1. The requirement, and the one part of it that cannot be met

The requirement is that Circle can freeze USDC balances, while USDC otherwise behaves normally on the DEX and on Lightning; that a blacklisted holder's channels stop working and their resting orders never complete; and that breakage is clean, harming no one else.

Everything in that is achievable except one thing, and it is worth stating before the design so the design is read correctly.

A consensus rule cannot freeze a Lightning balance. A channel's funds sit in a single 2-of-2 output that holds *both* parties' claims, and the split between them exists only in off-chain state. Consensus sees one output and cannot tell whose money is inside. That leaves exactly two options and no third:

- **Do not freeze the funding output.** Then a frozen holder with a channel is not frozen at all: they can move their whole balance off-chain, where no consensus spend occurs, or co-sign a close paying everything to a fresh script.
- **Freeze the funding output.** Then the innocent peer's entire balance is trapped permanently, with no unilateral escape, because every exit from a channel spends that one output.

Related, and equally unfixable at the consensus layer: an issuer cannot tell from chain data whether

freezing a holder's `to_local` output is legitimate or is destroying the peer's penalty claim against a cheat, because the two are byte-identical and the difference is private off-chain data. And because Lightning's remedies are races against absolute deadlines, a *temporary* freeze on a shared script does not delay anyone; it silently hands the disputed value to the frozen party.

So Lightning is not a gap in this proposal to be closed later. It is the boundary of what consensus can do, and the proposal is built around it rather than pretending otherwise.

2. What consensus does

2.1 A supervised asset is declared at birth

The issuance commits an **issuer freeze key** into the asset id. Freezability therefore cannot be added to an existing asset or removed from a supervised one, and a freeze record is valid only if signed by that key. Ordinary assets are untouched by every rule below.

2.2 Enforcement is on spending, and only for single-owner outputs

Consensus rejects a transaction that *spends* a supervised-asset output when two things hold: the output's script is frozen, and the spend reveals a **single-owner** script (a key-path or single-signature spend). Creating outputs that pay a frozen script stays legal.

Both halves matter, and each removes a specific way the naive rule destroyed third-party funds:

- **Spend-only** means an innocent party can always be *paid*. Nobody's incoming funds can be blocked because their counterparty is frozen.
- **Single-owner-only** means a freeze can never bite on a script that holds someone else's claim. A 2-of-2, an HTLC, a covenant with a refund branch: the spend reveals the script, consensus sees it is not single-owner, and the freeze does not apply. This is decidable at spend time even though it is undecidable at freeze time, because the witness reveals the script that the address only committed to as a hash.

The cost of that second rule is explicit: a holder who moves supervised funds into a shared script before being frozen has escaped the consensus rule. That is the same escape as moving funds to an exchange or a rollup on Ethereum, and it is why section 3 exists.

2.3 Resting DEX orders are covered, and cleanly

A covenant order is the good case, and it is worth separating from Lightning because the difference is instructive: a resting covenant output holds **only the maker's own asset**, so freezing it traps nobody else. The order's script is publicly derivable from the order parameters, so an issuer can freeze it. A taker who tries to fill afterwards has their transaction rejected and loses nothing but the attempt.

For this to be clean rather than merely safe, the DEX must reap frozen orders from the book, which needs a signal it does not have today (4.2).

2.4 Supervised assets are explicit

Consensus cannot see which asset a blinded output holds, so a supervised asset must never be blinded. This is not free, and the cost is in a place worth naming precisely: **the node wallet currently blinds change automatically** whenever a transaction has two or more change outputs, which under any-asset fees is the ordinary case for sending an issued asset. Shipping the rule without fixing that would break ordinary sends. Both the node wallet and the DEX's own wallet daemon need the change to keep supervised change explicit.

3. What the Lightning and DEX implementations must do

This is the other half of the design, not an afterthought, and it is where a blacklisted holder's channels actually stop working.

- **Refuse to open** a supervised-asset channel with a blacklisted peer.
- **Refuse to forward or route** supervised-asset payments for a blacklisted peer, which is what makes the channel stop working in practice.
- **Close on blacklist.** On learning a peer is frozen, initiate a close. The commitment pays the frozen party's own script, so the balance lands on-chain where consensus can hold it, and the innocent peer is paid out normally and can spend immediately.
- **Never freeze the funding script**, as an operational rule for the issuer. Consensus already declines to apply a freeze to a shared script under 2.2, so this is defence in depth rather than the only line.
- **DEX: reap frozen orders** from the book so takers never attempt a fill that consensus will reject.

The honest consequence: between a holder being blacklisted and their channels closing, they can still move value inside Lightning. That window is a policy matter for the Lightning implementation, not something consensus can shorten.

4. What must ship with the rule

4.1 Mempool eviction

A spend that was valid on entry becomes invalid the moment a freeze confirms, and nothing removes it today: the mempool drops only transactions included in a block or directly conflicting, and a freeze record conflicts with nothing. The stale transaction is re-selected into every block template, template construction fails, and producers skip slots — a chain halt, not an inconvenience. The node also asserts this cannot happen: `CTxMemPool::check` re-runs `CheckTxInputs` inside a bare `assert` commented "should always pass" (`src/txmempool.cpp:920`), so a freeze would abort nodes. Eviction on freeze, re-admission on unfreeze, and a reconsidered assert are part of the feature, not follow-up work.

4.2 A freeze signal tools can read

An RPC answering "is this asset supervised" and "is this script frozen", plus a distinct machine-readable rejection reason. Without it the DEX cannot reap orders, wallets cannot explain a failure, and the Lightning implementation cannot act on a blacklist at all. Section 3 is unimplementable without this.

4.3 Wallet change handling

Per 2.4: keep supervised-asset change explicit in the node wallet and in the DEX's wallet daemon. This is the item most likely to be discovered late and to look like an unrelated bug.

4.4 Activation and cutover

Height-gated above the tip at release, with the same all-nodes-at-once discipline as the Simplicity activation and the 60-second-block fork, because an un-upgraded node diverges at the gate.

5. Limits, stated so nobody discovers them later

- **It freezes coins at a script, not a person.** A holder spread across twenty addresses must be frozen twenty times, and the freezing transaction is public before it confirms, so an attentive target can move first. Against a custodian, an exchange deposit address or a wallet identified by analysis it works; against an evading self-custody holder it is a race.
- **Shared scripts are outside its reach**, by design, because the alternative is trapping innocent counterparties.
- **Off-chain balances are outside its reach**, permanently. Lightning, and any future venue holding pooled balances, must enforce at their own layer.
- **It is not retroactive**, and must not be: a rule that rejects spends cannot apply to history produced before it existed.

6. What this means for USDC

USDC.e has zero supply and zero deposits, and supervision can only be conferred at issuance, so the choice is open now and closes the moment real USDC is bridged.

My recommendation: **build the capability, and decide USDC's status with Circle rather than ahead of them.** A supervised USDC gets Circle the freeze it has everywhere else, keeps working on the DEX, and keeps working on Lightning while unfrozen, with the caveat that a blacklisted holder's Lightning balance is reached by the Lightning implementation rather than by consensus. That is a defensible answer to give them, and it is materially better than the one we have today, which is no freeze at all.

7. For Alberto

1. **Single-owner detection at spend time.** The rule turns on classifying a revealed script as

single-owner. P2WPKH and P2TR key-path are clear. Is a bare P2PK or a 1-of-1 multisig worth including, and where should the classifier live so that mempool and block validation cannot disagree?

2. **Where the freeze key is committed.** Folding it into the existing contract hash is the smaller change; a distinct derivation constant makes the class visible from the id alone. The second is more honestly "identifiable".
3. **Mempool eviction design.** This is the piece most likely to go wrong, and it touches code we both rely on. I would want your review of the eviction and re-admission paths specifically.
4. **Whether supervised assets may pay fees,** given a frozen holder's fee output interacts with producer fee handling.

8. Provenance

This design is what survived ten independent analyses run against the real trees, whose blocker-rated findings were then attacked by separate reviewers. Two earlier shapes were discarded by that process: freezing arbitrary scripts of any type, which strands innocent counterparties and converts Lightning's timelock races into thefts; and restricting supervised assets to single-owner outputs entirely, which would have banned them from the DEX and Lightning outright and cost more than it bought.

Claims about the mempool assert and about whole-transaction rejection were re-verified by hand. One reported finding was discarded as an artifact of the shared `~/Sequentia` checkout, which sits on `pos-exprace` and still shows Simplicity inactive while master has it active and testnet proved it in block 89860; that checkout should be brought up to date, since other sessions read it as ground truth.

Prepared by Saba. No consensus code has been written; a sketch produced during analysis was reverted unbuilt.